

MOS²: A Two-Stage Multi-Objective Framework for Server Deployment and Service Provisioning in Mobile Edge Computing

1st Haiming Liu
School of Software Engineering
Beijing Jiaotong University
Beijing, China
liuhaiming@bjtu.edu.cn

2nd Zecheng Xu
School of Software Engineering
Beijing Jiaotong University
Beijing, China
21301055@bjtu.edu.cn

3rd Xiaofei Di,
School of Software Engineering
Beijing Jiaotong University
Beijing, China
xfdi@bjtu.edu.cn

4th Jingxin Su
School of Software Engineering
Beijing Jiaotong University
Beijing, China
jxsu@bjtu.edu.cn

5th Xiaoping Che
School of Software Engineering
Beijing Jiaotong University
Beijing, China
xpche@bjtu.edu.cn

Abstract—Edge computing has emerged as a promising paradigm for supporting latency-sensitive and computationally intensive applications. In this context, effective server deployment and service provisioning are crucial for optimizing system performance and maximizing platform profit. This paper addresses the joint problem of server deployment and service provisioning in multi-user scenarios, with the goal of minimizing overall cost and user access latency while satisfying constraints such as distance thresholds, resource capacities, and connectivity requirements. To this end, we propose a novel two-stage framework that decouples the problem into two interrelated but relatively independent stages. In the first stage, we aim to minimize both deployment cost and transmission cost by proposing a cost-benefit-driven server deployment strategy. This strategy considers the geographical distribution of users and base stations to determine an appropriate number of servers based on a cost budget and user density. We formulate the first stage as a K-Median-inspired problem and design a Coverage-based Local Search (CLS) algorithm to determine the specific server locations. In the second stage, we propose a Pareto-oriented Service Provisioning (PSP) strategy that couples a hybrid-greedy initialization with NSGA-II, jointly optimizing provisioning cost and service latency. Extensive experimental results demonstrate that the strategy effectively reduces deployment expenses and user response delays, offering improved service quality and enhanced economic benefits for service providers and network operators.

Index Terms—mobile edge computing, multi-stage Pareto-based optimization, server deployment, service provisioning.

I. Introduction

With the continuous increase in the number of mobile users, the emergence of new applications and the

growing demand for portable services [have introduced](#) significant computational and communication burdens. Mobile Edge Computing (MEC), as an innovative computing paradigm, enables the offloading of computation-intensive tasks from resource-constrained mobile devices to edge servers situated near end users, which effectively alleviates the computational load on mobile devices and the core network, while addressing the issue of high-latency communication. However, the high deployment cost and limited computational capacity of edge servers, coupled with the heterogeneity of service requests from multiple end users, present a series of challenges to improving the profit margins of mobile network operators.

To address these issues, this paper proposes a multi-stage Pareto-based optimization strategy for edge server deployment and service provisioning that jointly takes into account the total deployment cost and user response time. [We aim to balance cost and latency while enabling efficient deployment of edge servers and services, enhancing network performance, improving resource allocation, and reducing edge-server operating cost.](#)

A. Motivation and Challenges

Server deployment and service provisioning have a significant impact on the profit of mobile network operators. [Figure 1 illustrates a scenario with five base stations. The solid black contours partition the topology into interconnected service regions, the black dashed circles denote the coverage areas of individual base stations, and the red dotted circles identify base stations equipped with edge](#)

Xiaofei Di is the corresponding author. Email: xfdi@bjtu.edu.cn.

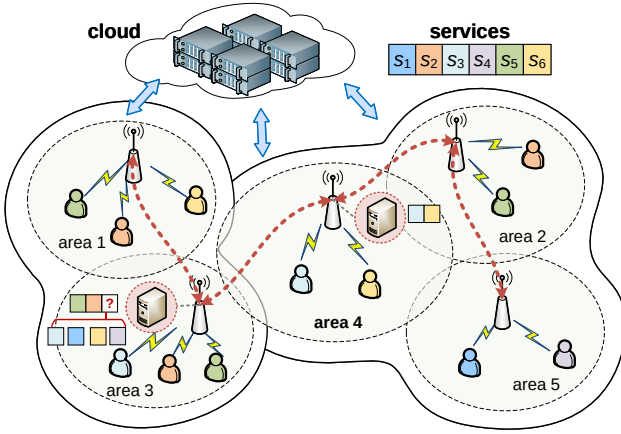


Fig. 1. MEC system model and service-provisioning scenario.

servers. The blue bidirectional arrows represent cloud-edge communication, while the red dashed arrows indicate inter-region request forwarding. Distinct colors represent service types s_1 – s_6 , and each user color indicates the corresponding requested service. The service blocks beside an edge server show its instantiated services; the question-mark slot in Area 3 denotes a provisioning decision under limited server capacity. Each service replica serves a user request and generates income. Because an edge server can host only a subset of the service catalog, a request without a local replica is forwarded to another server that provides the requested service, incurring additional transmission cost. For example, the server in Area 4 provisions services s_3 and s_6 ; requests for the other service types require inter-area forwarding. This setting gives rise to the following two challenges:

(i) Determining the optimal number and location of edge servers to maximize user coverage across diverse regions while minimizing the overall deployment cost for mobile network operators remains a critical challenge. As illustrated in Figure 1, deploying only a single server results in the lowest deployment cost but fails to cover all users, while deploying one server in each of the five areas incurs prohibitively high expenses. In this paper, we consider the spatial distribution of user densities and strategically deploy servers only in areas 3 and 4. Leveraging the connectivity among base stations, this deployment strategy provides service to the maximum number of users at a moderate total cost. Through this approach, we achieve a balance between server deployment cost and transmission cost.

(ii) Designing an effective service provisioning strategy under constrained server capacity while jointly optimizing

provisioning cost and users' delay remains a significant challenge. For example, the server in Area 3 can host three service instances. Two slots are occupied by the illustrated service replicas, while the remaining slot must be assigned from the candidate services shown below the server. Since this choice affects forwarding distance, transmission cost, and service-delivery latency, the provisioning decision considers request frequency and service provisioning cost. Therefore, an appropriate combination of server deployment and service provisioning decisions is crucial for maximizing the profit of mobile network operators.

B. Contributions and Paper Organization

In this paper, we investigate the problem of server deployment and service provisioning in a multi-user scenario, aiming to jointly optimize the cost and user access latency under multiple constraints such as distance thresholds, resource limitations, and connectivity requirements. Our research contributions are summarized as follows:

- We develop a joint server-service optimization model to address the challenges of server deployment and service provisioning in multi-user scenarios. To better tackle this problem, we use a decomposition that separates it into two sequential stages. The first stage, server deployment, focuses on servers, determining where edge servers should be deployed. The second stage, service provisioning, involves deciding the provisioning locations of services across the selected servers.
- We formalize the first stage server deployment as a k -median problem and introduce a Coverage-based Local Search (CLS) strategy to determine optimal server locations. Building upon the deployment of servers, the second stage addresses service provisioning through an enhanced strategy featuring improved initial population generation with high-quality solutions. This constitutes our proposed Pareto-oriented Service Provisioning (PSP) approach, achieving joint optimization of provisioning cost and service latency.
- We conducted extensive experiments to compare our strategy with several baselines based on a real-world base station dataset provided by Beijing Telecom. The results are shown from different perspectives to provide conclusions. Extensive experiments demonstrate the superior performance of the proposed algorithms.

The remainder of this paper is organized as follows. In Section II, we review related research. In Section III, we model the problem. In Section IV, we propose the server deployment and service provisioning algorithms. In Section V, we present the comparative experimental results. Finally, in Section VI, we conclude the paper.

II. RELATED WORK

In recent years, a substantial body of research in mobile edge computing (MEC) has focused on either reducing deployment cost or minimizing user latency. However, few studies address the joint optimization of both objectives. As a result, it is often difficult to simultaneously satisfy both operator profitability and user quality of service. To bridge this gap, this paper investigates the cost-latency joint optimization of edge server deployment and service provision.

A. Server Deployment

Various methods have been proposed to solve the edge server deployment problem. Wang et al. [1] used mixed-integer programming with theoretical guarantees to obtain optimal placements aimed at minimizing access latency and balancing system loads. Li et al. [2] developed a particle swarm optimization-based layout algorithm for energy-efficient server deployment. Xu et al. [3] proposed a load-aware deployment strategy that leverages decision trees and NSGA-II to construct balanced layouts. Jiang et al. [4] introduced two deep reinforcement learning-based algorithms for dynamic server placement. Kasi et al. [5] utilized genetic algorithms to explore large solution spaces for cost-minimizing deployment. Wu et al. [6] proposed three greedy heuristics, and Shen et al. [7] designed a dynamic deployment method. Ling et al. [8] formulated the edge server placement problem as a constrained optimization problem and applied the Particle Swarm Optimization (PSO) algorithm to optimize network latency and energy consumption jointly. Luo et al. [9] proposed an edge server placement algorithm named DQN-ESPA, which leverages Deep Q-Networks and reinforcement learning to achieve optimal placement without relying on prior deployment experience. Zeng et al. [10] developed a global optimization method based on simulated annealing and a greedy heuristic to satisfy capacity constraints of edge servers while minimizing the number of simultaneously active edge servers, thus enabling efficient and cost-effective deployment. Guo et al. [11] proposed an approximate method using k-means and mixed-integer quadratic programming to balance the workload among edge clouds. Xu et al. [12] studied the joint deployment and allocation of virtual network functions, aiming to achieve a balance between cost and QoS. Cui et al. [13] modeled this joint user coverage and network robustness oriented k-edge server placement problem and showed that finding the optimal solution to the problem is NP-hard. However, most of these server deployment strategies focus primarily on energy efficiency and load balancing, with limited consideration of user distribution. This lack of user-centric design leads to placement decisions that may overlook demand-side char-

acteristics, potentially reducing the overall effectiveness of deployment strategies.

B. Service Provision

For service provision, Qian et al. [14] proposed a privacy-aware provisioning framework combining a centralized greedy algorithm with distributed federated learning, enabling both privacy protection and efficient service delivery through mobile edge clouds. Yu et al. [15] explored collaborative service provision and proposed a distributed algorithm based on matching theory to jointly optimize service placement and user association. Ma et al. [16] investigated long-term cost-constrained service layout and introduced a T-slot frame-based predictive provisioning algorithm to enhance system performance. Ouyang et al. [17] applied Lyapunov optimization to transform long-term constraints into real-time decision problems. Furthermore, they proposed an adaptive user-aware management mechanism that jointly optimizes perceived latency and service migration costs [18]. Zhang et al. [19] built a dynamic framework using control theory and game-theoretic modeling. Skarlat et al. [20] proposed a fog computing framework utilizing genetic algorithms, while Wang et al. [21] developed an online approximation algorithm with polynomial complexity for real-time service provisioning. Wang and Jiao [22] presented an iterative algorithm for collaborative edge service deployment. Cabrera C et al. [23] proposed a mobility-aware, priority-driven, ACO-based service deployment model that aims to minimize latency for critical applications by predicting the mobility paths of users. However, this model sacrifices load balancing among network servers. Yang et al. [24] leveraged user mobility patterns and service access behaviors to predict the future distribution of service requests, and dynamically adjusted service placement and load scheduling based on these predictions to reduce latency caused by accessing remote cloud servers. Chen et al. [25] investigated the problem of edge service deployment for application service providers under limited budget constraints, and they proposed a spatio-temporal edge service placement algorithm to enhance the performance of edge computing. Han et al. [26] addressed energy-saving and flexible service layout in mobile edge cloud networks supporting cell scaling. Sami et al. [27] proposed a deep reinforcement learning solution for fast and proactive service updates to meet user needs. However, most of these strategies assume homogeneous service requests or focus solely on satisfying user demands, without adequately accounting for the heterogeneity of task requirements. This results in inefficient resource allocation and ignores the profitability concerns of service providers.

Optimization and learning methods also address complementary MEC resource-management objectives. Feng et

al. [33] jointly optimized latency-aware service deployment and peer offloading over multiple timescales. Kato et al. [34] applied breakout local search to transmission- and processing-aware task offloading in a multi-tier cloud environment. Jing et al. [35] studied long-term max-min fairness for task splitting and resource allocation in integrated multi-RAT/MEC networks, while Jing et al. [36] used online mini-batch learning for dynamic energy-cost conservation in distributed edge clouds. These studies address dynamic offloading, fairness, or energy management; the present work focuses on interpretable server planning and capacity-constrained multi-objective service provisioning under a common cost-latency model.

To address the limitations outlined above, we propose a joint optimization framework for edge server deployment and service provision. The proposed strategy aims to simultaneously reduce deployment costs and user latency, thereby enhancing network performance and resource efficiency in MEC environments.

III. Model and Problem Formulation

In this paper, we focus on an optimization problem involving server deployment and service provisioning in a mobile edge computing environment. To jointly minimize latency and cost while satisfying constraints such as storage capacity and transmission bandwidth, we construct a comprehensive system model. Our objective is to optimize resource allocation and service provisioning strategies, considering constraints on resources and geographical distance.

A. The MEC Environment

In this paper, we consider a **hierarchical** network architecture comprising the cloud data center, edge servers, and mobile end users, as illustrated in Figure 1. The cloud layer serves as a centralized resource pool that hosts all services required by user requests. We use $\mathcal{S} = \{s_w\}$ to denote the set of services that are requested by users. For each service s_w , the consumption of storage resources is indicated by v_w , and the required computing resource is denoted by ς_w . At the edge layer, each edge server is assumed to be co-located with a base station and is responsible for provisioning services to support user access and process tasks near the user side. The set of base stations is denoted by $\mathcal{B} = \{b_k\}$, where b_k represents the k -th base station. Mobile network operators deploy edge servers according to the locations of base stations. The set of edge servers is denoted by $\mathcal{M} = \{m_j\}$, where m_j represents the j -th server. Let V_j denote the storage capacity of edge server m_j , indicating the maximum amount of storage resources available for service deployment. The set of services deployed on m_j is represented by S_j . At the user layer, the set of mobile users is denoted by $\mathcal{U} = \{u_i\}$, where u_i represents the i -th user. Each user generates a service

request u_{iw} , and multiple users may request the same type of service, which is represented by the set $\mathcal{U}_w = \{u_{iw}\}$. Key notations are summarized in Table I for clarity.

B. Cost Model

Mobile network operators are responsible for deploying edge servers and provisioning services to accommodate user requests, resulting in costs associated with server deployment, service provisioning, and data transmission.

1) **Server Deployment Cost:** The server deployment cost refers to the inherent cost incurred when deploying an edge server co-located with a base station. We assume that the servers are heterogeneous and that the deployment cost of server m_j is denoted by p_j . Let y_{jk} be a binary server-deployment variable, where $y_{jk} = 1$ if edge server m_j is deployed at base station b_k , and $y_{jk} = 0$ otherwise. Accordingly, the deployment cost of server m_j is $C_d(m_j) = y_{jk}p_j$.

2) **Service Provisioning Cost:** The service provisioning cost is represented by the set $\mathcal{SC} = \{sc_w\}$, where sc_w denotes the provisioning cost of service s_w . Let z_{jw} be a binary service-provisioning variable, where $z_{jw} = 1$ if an instance of s_w is provisioned on edge server m_j , and $z_{jw} = 0$ otherwise. Furthermore, let a_{ijw} be a binary association variable, where $a_{ijw} = 1$ if user u_i is served by service s_w on server m_j , and $a_{ijw} = 0$ otherwise. The total provisioning cost of server m_j is $C_p(m_j) = \sum_{s_w \in \mathcal{S}} z_{jw}sc_w$.

3) **Transmission Cost:** The transmission cost is produced by the physical distance between users and their assigned servers during network communications. Each user is associated with a server that provides the requested service, and a per-unit distance transmission cost coefficient is defined accordingly. Specifically, we assume that if a user is located within the optimal service coverage radius of its assigned server and the server can provide the requested service, the per-unit transmission cost coefficient is set to be 0, which indicates no transmission cost. If the user is located outside the optimal coverage area but the assigned server can still provide the requested service, the coefficient is set to the base value. If the assigned server cannot provide the requested service, the coefficient increases proportionally with the number of intermediate servers traversed during service routing. Finally, if no available service instance can be found after searching the entire area, the coefficient reaches its maximum value, which is defined as k times of the base value, where k corresponds to the total number of servers. Consequently, the transmission cost is modeled as the product of the physical distance between users and edge servers and the corresponding per-unit distance transmission cost coefficient, which is defined as $C_t(m_j) = \sum_{i \in \mathcal{U}} \alpha \cdot \beta_{iw} \cdot \tau(u_i \rightarrow m_j)$. Here, we use α to denote the unit price of communication bandwidth, β_{iw} represents the number of hops between the location of user u_i and the edge server providing service

TABLE I
SUMMARY OF SYMBOLS IN THE SYSTEM MODEL

Symbol	Definition
$\mathcal{U}$	Set of users, $U = \{u_i\}$
$\mathcal{M}$	Set of servers, $M = \{m_j\}$
$\mathcal{S}$	Set of services, $S = \{s_w\}$
$\mathcal{B}$	Set of base stations, $B = \{b_k\}$
$\mathcal{SC}$	Set of service deployment costs, $SC = \{sc_w\}$
p_j	Deployment cost of server m_j
a_{ij}	Indicator whether user u_i is served by m_j .
y_{jk}	Indicator whether server m_j is deployed at base station b_k .
a_{ijw}	Indicator whether user u_i is served by s_w on m_j .
z_{jw}	Indicator whether server m_j provisions service s_w .
C_{max}	Maximum allowed total server deployment cost
$\mathcal{K}^c$	Maximum number of deployable server nodes under C_{max}
σ_{min}	Minimum required user density
$\mathcal{K}^r$	Number of base stations with local user density exceeds σ_{min}
k	Number of servers actually deployed
α	Unit price of communication bandwidth
β_{iw}	Number of hops between user u_i and server m_j providing s_w .
$\tau(u_i \rightarrow m_j)$	Distance between user u_i and server m_j
V_j	Service capacity of server m_j
R_{iw}^c	Required computation resource for user u_i requesting s_w
R_{iw}^t	Data volume transmitted by user u_i
μ_j	Computing capacity of server m_j
r_{ij}	Maximum transmission rate from u_i to m_j
D_i	Maximum tolerable transmission delay of u_i

s_w , and $\tau(u_i \rightarrow m_j)$ denotes the physical distance between user u_i and m_j .

Therefore, the total cost for services provided by server m_j is defined as follows:

$$\mathbb{C}(m_j) = C_d(m_j) + C_p(m_j) + C_t(m_j). \quad (1)$$

C. Latency Model

In this system, user latency occurs during both computation and communication. We define $D_p(m_j)$ as the computational latency associated with user requests processed by edge server m_j . This latency depends on the resource requirements of the requested services and the computing capabilities of the servers, and is given by $D_p(m_j) = \sum_{i \in \mathcal{U}} a_{ijw} R_{iw}^c / \mu_j$, where μ_j denotes the computing capacity of edge server m_j , R_{iw}^c represents the computational resource demand of service s_w requested by user u_i , and a_{ijw} indicates whether u_i is served by s_w on m_j .

In addition, communication latency refers to data transmission between users and edge servers when a requested service is provisioned outside the user's region. It is influenced by the propagation and transmission times of

the network, where the propagation delay depends on the distance $\tau(u_i \rightarrow m_j)$ between user u_i and edge server m_j hosting service s_w . We define the maximum transmission rate as $r_{ij} = B_{ij} \log_2(1 + \gamma g_{ij} / \mathcal{N})$ [28], [31], [32], where B_{ij} denotes the channel bandwidth, γ is the transmission power of the user device, $\mathcal{N}$ is the noise power, and g_{ij} is the channel gain between u_i and m_j . The channel gain is modeled as $g_{ij} = 127 + 30 \log(\tau(u_i \rightarrow m_j))$. For each user, the transmitted data volume is denoted by R_{iw}^t . When a requested service is unavailable at the nearest server, the request is forwarded through intermediate servers; β_{iw} captures the resulting per-hop communication delay. Consequently, the communication latency is $D_c(m_j) = \sum_{i \in \mathcal{U}} a_{ijw} (R_{iw}^t / r_{ij}) \beta_{iw}$. Therefore, the total latency for users provided by server m_j is defined as follows:

$$\mathbb{D}(m_j) = D_p(m_j) + D_c(m_j). \quad (2)$$

D. Problem Formulation

On the basis of the models above, our objective is to design an efficient strategy that jointly minimizes the total cost and latency, while considering user mobility under the system constraints. Thus, the optimization objective is formulated as follows:

$$P_1 : \text{minimize} \quad \sum_{m_j \in \mathcal{M}} \mathbb{Q}(\mathbb{D}(m_j), \mathbb{C}(m_j)) \quad (3)$$

$$\text{s.t.} \quad \sum_{m_j \in \mathcal{M}} C_d(m_j) \leq C_{\max}, \quad (4)$$

$$\sum_{w \in \mathcal{S}} z_{jw} v_w \leq V_j, \quad \sum_{w \in \mathcal{S}_j} z_{jw} s_w \leq \mu_j, \quad (5)$$

$$a_{ijw} (R_{iw}^c / \mu_j + (R_{iw}^t / r_{ij}) \beta_{iw}) \leq \widetilde{D}_i, \quad \forall i \in \mathcal{U}, \quad (6)$$

$$y_{jk}, z_{jw}, a_{ijw} \in \{0, 1\}, \quad \forall i \in \mathcal{U}, \forall j \in \mathcal{M}. \quad (7)$$

P_1 is the objective function, and Eqs. (4)–(7) specify its constraints. Equation (4) limits total server-deployment cost to budget $C_{\max}$. Equation (5) enforces the storage and computing capacities of each edge server. Equation (6) provides latency-oriented QoS by requiring the end-to-end latency of a served request to remain below the user's tolerable bound $\widetilde{D}_i$. For reliability-critical applications, latency compliance can be complemented by packet-loss, link-availability, and service-interruption constraints, preventing a low-latency solution from being selected when service continuity is insufficient. Equation (7) defines the server-deployment variable y_{jk} , service-provisioning variable z_{jw} , and user-service association variable a_{ijw} . To solve this problem, we use a decomposition that separates it into two sequential stages. The first stage focuses on servers, determining where edge servers should be deployed. The second stage addresses providing services, i.e., deciding the provisioning locations of services across the selected servers.

The decomposition follows the operational hierarchy of MEC planning. Server deployment is a long-term infrastructure decision governed by budget, coverage, and geographic demand, whereas service instances are provisioned after the physical server locations are known. Stage 1 therefore resolves the spatial variables that determine deployment and transmission costs, and Stage 2 optimizes service instances and user associations on the selected infrastructure. Passing the selected locations, user assignments, and Stage-I cost to Stage 2 preserves the cost-latency dependence while substantially reducing the decision space of the service-provisioning search.

In the first stage (server deployment), we isolate the cost terms $C_d(m_j)$ and $C_t(m_j)$ from the original problem P_1 and consider them as the primary objective for this stage, since both are directly influenced by the locations of the deployed edge servers. The trade-off in the first stage is that reducing the deployment cost $C_d(m_j)$ may lead to increased transmission cost $C_t(m_j)$, and vice versa. Therefore, our objective is to balance these two costs by solving the following optimization problem:

$$P_2 : \quad \text{minimize} \quad \sum_{m_j \in M} C_d(m_j) + C_t(m_j) \quad (8)$$

$$\text{s.t.} \quad (4), (7) \quad (9)$$

According to the above formulation, this stage aims to determine the server deployment strategy, including both the number and locations of edge servers to be deployed. The problem is NP-hard and can be reduced to the classical set cover problem [29], [30].

In the second stage (service provisioning), we address the remaining portion of the original problem P_1 . The resulting cost value $\mathbb{C}$ from the first stage is treated as a fixed input in the second stage, which focuses on the joint optimization of the remaining provisioning cost $C_p(m_j)$ and service latency $\mathbb{D}(m_j)$. The objective function is defined as follows:

$$P_3 : \quad \text{minimize} \quad \sum_{m_j \in M} Q(\mathbb{D}(m_j), C_p(m_j), \tilde{\mathbb{C}}) \quad (10)$$

$$\text{s.t.} \quad (5) \sim (7) \quad (11)$$

Therefore, we are concerned with the cost of service provisioning and user-perceived latency by designing a strategy that jointly optimizes the remaining provisioning cost and latency in a fixed server configuration.

IV. Algorithm Design

In this section, we present a staged decision-making process that captures server deployment decisions based on user density distribution and operator budget, and supports a latency and cost-aware service provisioning mechanism. Our strategy aims to minimize deployment and transmission costs during these two stages from the perspective of mobile network operators, and to jointly

optimize the provisioning cost and user-perceived latency in the subsequent service provisioning stage.

To systematically illustrate our proposed approach, Fig. 2 depicts the overall architecture of the multi-stage joint optimization framework. The problem is logically decoupled into two primary interconnected stages:

1) Stage 1: CLS-based Server Deployment. Building on the system model established above, Stage 1 is triggered to determine the optimal placement of edge servers. Initially, the exact number of deployable servers is rigorously derived by jointly evaluating the operator's budget constraints and the user density requirements. Subsequently, a Coverage-based Local Search (CLS) algorithm is executed among the candidate base stations. Through an iterative "Swap" mechanism, the algorithm continuously evaluates and minimizes the total deployment and transmission cost. The loop terminates when no further cost decrease can be achieved, outputting a fixed set of deployed server locations. This spatial configuration acts as a fundamental physical constraint for the subsequent phase.

2) Stage 2: PSP Strategy for Service Provisioning. Given the fixed server locations, PSP constructs a hybrid initial population and applies NSGA-II to optimize provisioning cost and access delay. Crossover and mutation produce each offspring, whose entries are first rounded to binary values. For every server m_j , PSP then checks $\sum_w z_{jw} \leq V_j$. If the capacity is exceeded, selected service entries are randomly deactivated until feasibility is restored. Only capacity-feasible offspring are evaluated and passed to non-dominated sorting; thus, feasibility is enforced by repair rather than by a penalty function. The evolutionary process returns a set of provisioning schemes representing different cost-delay trade-offs.

A. Server Deployment Optimization: CLS

In the first stage, we focus on optimizing the deployment of edge servers to support diverse services in dynamic edge environments and to assist mobile network operators in making informed placement decisions. Considering the distribution of user demand and budget constraints, this stage seeks to identify efficient server locations that maximize service coverage and system performance.

We reformulate server deployment as a facility-location problem in which candidate base stations are potential server sites and users are demand points with heterogeneous service requirements. The deployment budget $C_{\max}$ gives the upper bound $\mathcal{K}^c = \lfloor C_{\max} / \min_j \{p_j\} \rfloor$ on the number of deployable servers. For each candidate base station b_k , we compute the user density σ_k within a circular coverage region of radius r . Let $\mathcal{K}^r = \sum_{b_k \in \mathcal{B}} \mathbf{1}_{\{\sigma_k \geq \sigma_{\min}\}}$ denote the number of candidate sites whose local density reaches threshold $\sigma_{\min}$. The target server count is then $k = \min\{\mathcal{K}^c, \mathcal{K}^r\}$, reflecting both budget feasibility and the

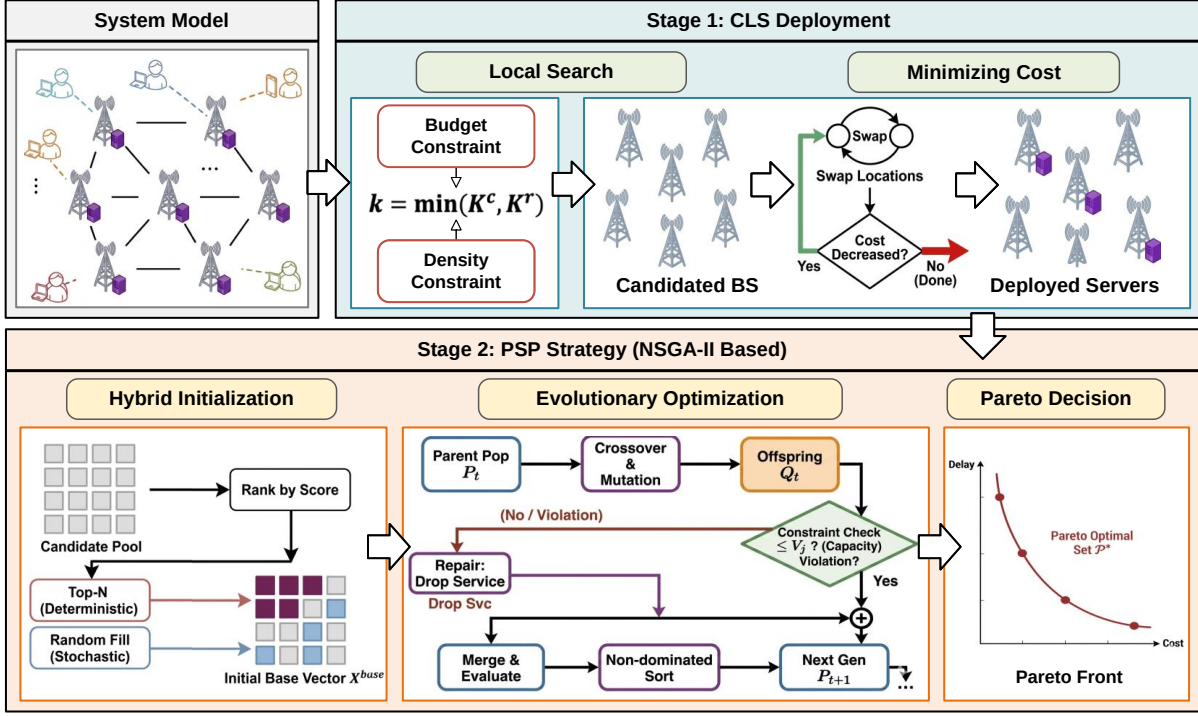


Fig. 2. Overall architecture of MOS².

spatial distribution of demand. Based on this estimate, we achieve an initial optimization of the deployment cost $C_d(m_j)$. We then optimize the transmission cost $C_t(m_j)$ and integrate it into a classical location problem by using the k -median formulation. Specifically, given a set of candidate server locations where each server can serve a certain number of users, the objective is to select k base stations that deploy the servers in such a way that the total distance between each user and the nearest server is minimized. The detailed steps are outlined in Algorithm 1.

To begin, several preparatory elements are required for the algorithm: the candidate server position set $\mathcal{M}$, representing all potential deployment locations; the user position set $\mathcal{U}$; the target number of servers k to be selected; the coverage radius r , used to calculate the service coverage area of each server; and the maximum number of iterations $T_{\max}$, which controls the termination condition of the algorithm.

In line 1, the algorithm randomly selects k positions from $\mathcal{M}$ to form the initial server deployment $\mathcal{S}$, and line 2 computes the corresponding communication cost C over $\mathcal{U}$ under the coverage radius r . Lines 4–19 then enter the iterative optimization phase. For each server position $i \in \mathcal{S}$, the algorithm attempts to replace i with a non-selected candidate $j \in \mathcal{M} \setminus \mathcal{S}$, generating a new solution $\mathcal{S}' = \mathcal{S} \setminus \{i\} \cup \{j\}$, and its communication cost

C' is calculated in line 9. In lines 10–14, if $C' < C$, indicating an improvement, the new solution is accepted and the current deployment and cost are updated as $\mathcal{S} \leftarrow \mathcal{S}'$ and $C \leftarrow C'$. Lines 17–18 then check whether any improvement was found in the current iteration; if no cost-reducing swap exists or $T_{\max}$ is reached, the algorithm terminates. Finally, in line 20, the algorithm returns the server deployment $\mathcal{S}$ with the minimum communication cost and the corresponding minimal cost C .

The computational complexity of Algorithm 1 is $O(T_{\max} \cdot k \cdot |\mathcal{M}| \cdot |\mathcal{U}|)$. Within each iteration, for every server slot $i \in \mathcal{S}$ the algorithm examines at most $|\mathcal{M}| - k$ candidate replacements $j \in \mathcal{M} \setminus \mathcal{S}$, and every invocation of $\text{ComputeCost}(\cdot)$ aggregates the coverage distances over all $|\mathcal{U}|$ users in $O(|\mathcal{U}|)$ time. The per-iteration cost is therefore $O(k \cdot |\mathcal{M}| \cdot |\mathcal{U}|)$, and the outer loop runs at most $T_{\max}$ times, yielding the stated bound. In practice the early-termination flag Δ exits the loop as soon as no cost-reducing swap can be found, so the observed runtime is substantially lower than this worst-case expression. Since $T_{\max}$ is a small constant and $k \ll |\mathcal{M}|$ in realistic MEC deployments, Algorithm 1 scales linearly in the product of the candidate-pool size and the user population, which guarantees tractability at city-scale.

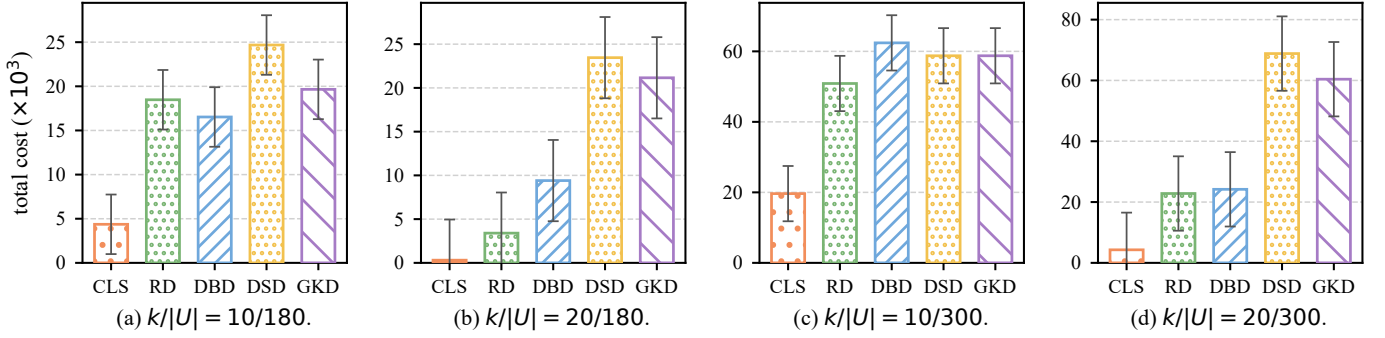


Fig. 3. Stage-I server-deployment performance under different server and user scales.

Algorithm 1: Coverage-based Local Search (CLS)

Input: $\mathcal{M}, \mathcal{U}, k, r, T_{\max}$
Output: $\mathcal{S}, C$

- 1 $\mathcal{S} \leftarrow \text{RandomSelect}(\mathcal{M}, k);$
- 2 $C \leftarrow \text{ComputeCost}(\mathcal{S}, \mathcal{U}, r);$
- 3 $t \leftarrow 0;$
- 4 while $t < T_{\max}$ do
 - 5 $\Delta \leftarrow \text{False};$
 - 6 foreach $i \in \mathcal{S}$ do
 - 7 foreach $j \in \mathcal{M} \setminus \mathcal{S}$ do
 - 8 $\mathcal{S}' \leftarrow \mathcal{S} \setminus \{i\} \cup \{j\};$
 - 9 $C' \leftarrow \text{ComputeCost}(\mathcal{S}', \mathcal{U}, r);$
 - 10 if $C' < C$ then
 - 11 $\mathcal{S} \leftarrow \mathcal{S}';$
 - 12 $C \leftarrow C';$
 - 13 $\Delta \leftarrow \text{True};$
 - 14 break;
 - 15 if Δ then
 - 16 break
 - 17 if not Δ then
 - 18 break
 - 19 $t \leftarrow t + 1;$
- 20 return $\mathcal{S}, C$

B. Pareto-based Service Provisioning - PSP

After determining the number and locations of deployed servers in the first stage, we proceed to address service provisioning on these servers to meet actual user demands. According to P_3 , the functions $\mathbb{D}(m_j)$ and $C_p(m_j)$ remain undetermined at this stage. Therefore, we propose a Pareto-based strategy to jointly optimize cost and delay by deploying diverse services to selected servers while satisfying capacity constraints, thereby solving the service provisioning problem under fixed server locations.

A service-provisioning scheme is encoded by a binary

Algorithm 2: Initial Base Configuration (IBC (α, β, ϖ_j))

Input: $\mathcal{M}, \mathcal{U}, \mathcal{S}, k, V_j, \alpha, \beta, \varpi_j$
Output: X^{base}

- 1 Initialize $X^{base} \leftarrow \mathbf{0};$
- 2 for $j \leftarrow 1$ to k do
 - 3 $C_j^{cost} \leftarrow \text{CostGreedyCandidates}(j);$
 - 4 $C_j^{delay} \leftarrow \text{DelayGreedyCandidates}(j);$
 - 5 Construct set $C_j \leftarrow C_j^{cost} \cup C_j^{delay};$
 - 6 foreach $s_w \in C_j$ do
 - 7 Calculate the hybrid score $H_j(s_w);$
 - 8 Sort C_j by descending $H_j(s_w);$
 - 9 if $|C_j| > V_j$ then
 - 10 $A_j \leftarrow \text{Top-N}(C_j, \varpi_j);$
 - 11 $R_j \leftarrow \text{Select}(C_j \setminus A_j, V_j - \varpi_j);$
 - 12 $L_j \leftarrow A_j \cup R_j;$
 - 13 else
 - 14 $L_j \leftarrow C_j;$
 - 15 Set row j of X^{base} to the indicator vector of L_j
- 16 return X^{base}

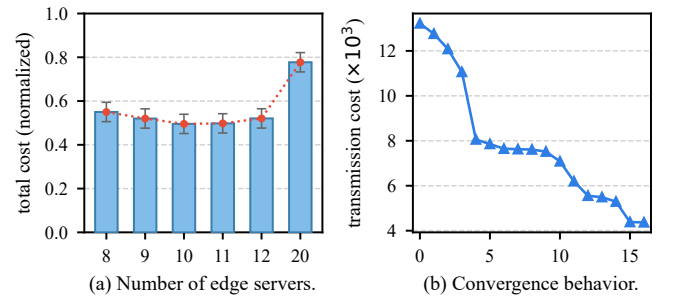


Fig. 4. Stage-I cost analysis under varying server counts and CLS iterations.

matrix $\text{indiv} \in \{0,1\}^{k \times |\mathcal{S}|}$. Each row corresponds to a

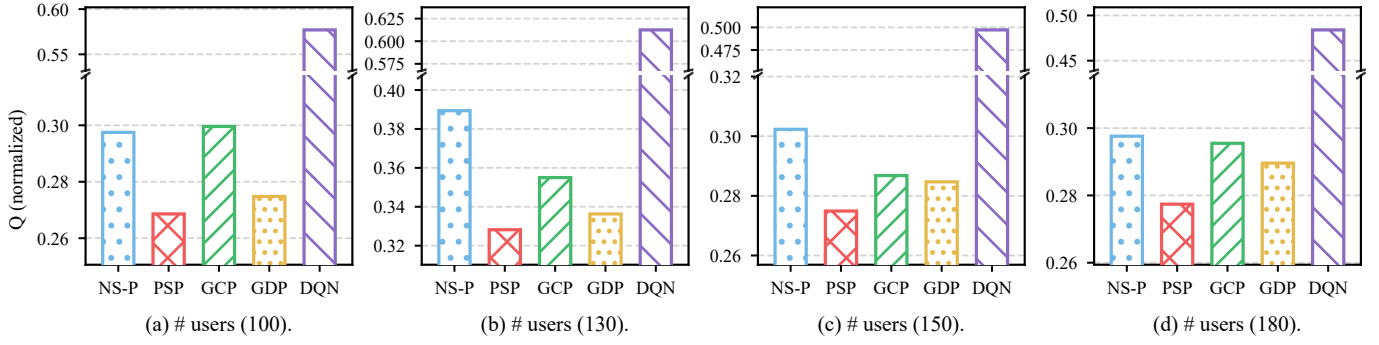


Fig. 5. Service-provisioning performance with 10 deployed servers and increasing user populations.

Algorithm 3: Pareto-based Service Provisioning (PSP)

Input: $\mathcal{M}, \mathcal{U}, \mathcal{S}, k, G, N, V_j, \alpha, \beta, \varpi_j$
Output: $\mathcal{P}^*$

- 1 $X^{base} \leftarrow \text{IBC}(\alpha, \beta, \varpi_j)$;
- 2 Generate initial population $\mathcal{P}$ of size N by stochastic bit-flip perturbation around X^{base} ;
- 3 foreach $X \in \mathcal{P}$ do
 - 4 Round and reshape X to $k \times |\mathcal{S}|$;
 - 5 For each row j , randomly deactivate selected entries while $\sum_w X_{jw} > V_j$;
 - 6 Evaluate objectives $C_p(m_j)$ and $\mathbb{D}(m_j)$;
- 7 for $g \leftarrow 1$ to G do
 - 8 Non-dominated sort $\mathcal{P}$ and assign crowding distances;
 - 9 Select parents via tournament selection;
 - 10 Apply crossover and mutation to produce offspring $\mathcal{P}'$;
 - 11 foreach $X' \in \mathcal{P}'$ do
 - 12 Round and reshape X' ; repair each violating row as above; evaluate X' ;
 - 13 $\mathcal{P} \leftarrow \text{Top-N}(\mathcal{P} \cup \mathcal{P}', N)$;
- 14 return $\mathcal{P}^*$

deployed server and each column to a service type. The entry $\text{indiv}[j, w]$ is the encoded counterpart of z_{jw} : it equals 1 when service s_w is provisioned on server m_j , and 0 otherwise. The provisioning cost on m_j is

$$C_p(m_j) = \sum_{w \in \mathcal{S}} \text{indiv}[j, w] s_{c_w}. \quad (12)$$

To ensure that no server exceeds its capacity, the encoding satisfies

$$\sum_{w \in \mathcal{S}} \text{indiv}[j, w] \leq V_j, \quad \forall j \in \mathcal{M}. \quad (13)$$

After crossover and mutation, every encoded entry is rounded to 0 or 1. If $\sum_w \text{indiv}[j, w] > V_j$ for any server j , selected entries in that row are randomly set to 0 until the capacity constraint is satisfied. Objective evaluation and non-dominated sorting are then performed on the repaired feasible individual.

PSP uses a hybrid initialization before the NSGA-II evolutionary loop. For each server, it constructs one ranking that favors low provisioning cost and another that favors services requested frequently by locally assigned users. The two rankings are fused by a hybrid score. High-scoring services form a deterministic anchor set, and the remaining capacity is filled by stochastic sampling from lower-ranked candidates. The resulting base configuration combines exploitation of high-quality candidates with exploration of alternative service combinations.

Definition 1 (hybrid score): Let $\alpha, \beta \in [0, 1]$ balance provisioning cost and local demand, with $\alpha + \beta = 1$. Let ϖ_j ($0 < \varpi_j \leq V_j$) denote the deterministic anchor size on edge server m_j . The hybrid score of candidate service s_w is

$$H_j(s_w) = \alpha w_{\text{cost},j}(s_w) + \beta w_{\text{req},j}(s_w), \quad (14)$$

where $w_{\text{cost},j}$ and $w_{\text{req},j}$ are descending rank scores derived from provisioning cost and the request frequency of users assigned to m_j , respectively. We set $\varpi_j = \lceil \rho V_j \rceil$ with $\rho = 0.5$, reserving half of the capacity for high-ranked deterministic services and half for stochastic selection from the remaining candidates.

Line 1 of Algorithm 2 initializes the base matrix X^{base} to a zero matrix, and the outer loop in lines 2–15 builds one row per deployed server. For each server j , lines 3–5 construct the candidate pool C_j by taking the union of two greedy rankings: the cost-oriented ranking C_j^{cost} returned by $\text{CostGreedyCandidates}(j)$ and the delay-oriented ranking C_j^{delay} returned by $\text{DelayGreedyCandidates}(j)$. Lines 6–7 then evaluate the hybrid score $H_j(s_w)$ defined in Eq. (14) for every candidate service $s_w \in C_j$, and line 8 sorts C_j in descending order of $H(\cdot)$. Lines 9–14 perform

the capacity-aware selection: if $|C_j|$ exceeds the per-server capacity V_j , the top ϖ_j services form the deterministic anchor set A_j (line 10, Top-N), while the remaining $V_j - \varpi_j$ slots are filled by randomly sampling from the lower-ranked remainder $C_j \setminus A_j$ (line 11, Select); otherwise all candidates are retained as $L_j = C_j$ (line 14). Line 15 converts L_j into an indicator vector and writes it to the j -th row of X^{base} , and line 16 returns the assembled base matrix, which serves as the hybrid seed for the subsequent evolutionary optimization.

Figure 6 illustrates the construction for Server 0. Panel (a) decomposes each candidate’s hybrid score into cost- and request-oriented components, and panel (b) shows the deterministic anchor, stochastic fill, and discarded candidates under the capacity limit.

During optimization, each individual represents a service deployment scheme encoded as a binary vector $X = [x_1, x_2, \dots, x_{k \cdot |\mathcal{S}|}]$ where $x_i \in \{0, 1\}$ indicates the deployment state at position i . Individuals are evaluated against two objectives: deployment cost $C_p(m_j)$ and access delay $\mathbb{D}(m_j)$. The algorithm first performs non-dominated sorting, prioritizing individuals by dominance rank. For equal-rank individuals, crowding distance comparison serves as the secondary selection criterion. Population updating employs classical crossover and mutation operators to expand the solution space. Crossover combines two parent individuals by exchanging gene segments at random points, while mutation randomly flips deployment states ($0 \leftrightarrow 1$) in vector X , abstracted as $X' = \text{mutation}(X)$.

Each generation merges the parent and offspring populations and retains N individuals according to non-dominated rank and crowding distance. At generation G , PSP returns the non-dominated capacity-feasible provisioning schemes found by the search. The detailed steps are outlined in Algorithms 2 and 3. The normalized score Q is applied after optimization to select one balanced solution for scalar comparison; it is not an objective used to generate the Pareto population.

Let $\mathcal{M}$ be the set of deployed servers obtained from Stage 1, let $\mathcal{U}$ be the set of user positions, let $\mathcal{S}$ be the service catalog, and let $k = |\mathcal{M}|$ be the number of deployed servers. Then, in Algorithm 3, line 1 invokes Algorithm 2 (IBC) to construct a base configuration X^{base} using our hybrid greedy strategy, and line 2 generates the initial population $\mathcal{P}$ of size N by binary perturbation around X^{base} . Lines 3–6 then round every individual, apply capacity repair, and evaluate its bi-objective values $C_p(m_j)$ and $\mathbb{D}(m_j)$. The main evolutionary loop (lines 7–14) performs non-dominated sorting with crowding-distance assignment, selects parents via tournament, applies crossover and mutation to produce offspring, repairs and evaluates them, and merges parent and offspring populations to retain the top N individuals for the next generation. Finally, the

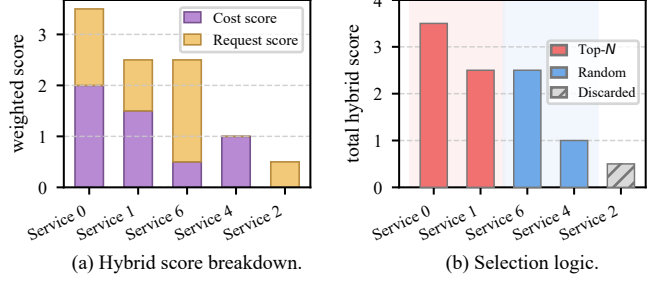


Fig. 6. Hybrid initialization for candidate services on Server 0.

Pareto-optimal set $\mathcal{P}^*$ is returned, representing the best cost–delay trade-offs found across all generations.

The overall complexity of the proposed PSP (Algorithm 3) is $O(G \cdot (N^2 + N \cdot (k \cdot |\mathcal{S}| + |\mathcal{U}|)))$, where G is the maximum number of generations and N the population size. The Initial Base Configuration (Algorithm 2) is executed exactly once before the evolutionary loop: for each of the k servers we build the cost-oriented and delay-oriented greedy rankings and evaluate the hybrid score $H(s_w)$ in $O(|\mathcal{S}|)$, sort the candidate set C_j by $H(\cdot)$ in $O(|\mathcal{S}| \cdot \log |\mathcal{S}|)$, and perform the Top- N anchor selection together with the random fill in $O(|\mathcal{S}|)$; therefore IBC contributes a one-time cost of $O(k \cdot |\mathcal{S}| \cdot \log |\mathcal{S}|)$. Inside the evolutionary loop, each generation performs non-dominated sorting and crowding-distance assignment in $O(N^2)$, constraint-aware repair and bi-objective evaluation in $O(k \cdot |\mathcal{S}| + |\mathcal{U}|)$ per individual, and elitist truncation in $O(N \cdot \log N)$. Because the IBC cost is strictly dominated by the evolutionary term under realistic population sizes, the hybrid initialization preserves the asymptotic complexity of a standard NSGA-II while empirically tightening the effective search space.

V. EXPERIMENTAL EVALUATION

A. Basic Setting

To evaluate the performance of our proposed CLS and PSP algorithms, we utilized a dataset obtained from China Telecom Company, with base station locations set within approximately a 9 km radius around Xizhimen Subway Station in Beijing. For the comparative experiments, we sequentially fixed the number of base stations and the scale of user requests to verify the effectiveness and superiority of the proposed algorithms under different data configurations. Regarding the system configuration, each server was assigned a coverage radius of 1.5 km, with computational capacities ranging from 2 GHz to 5 GHz, storage sizes from 5 GB to 10 GB, and communication bandwidths between 300 MB/s and 1000 MB/s. Each user submitted a service request belonging to a specific category, where each service type has fixed deployment costs and resource requirements. All experiments were

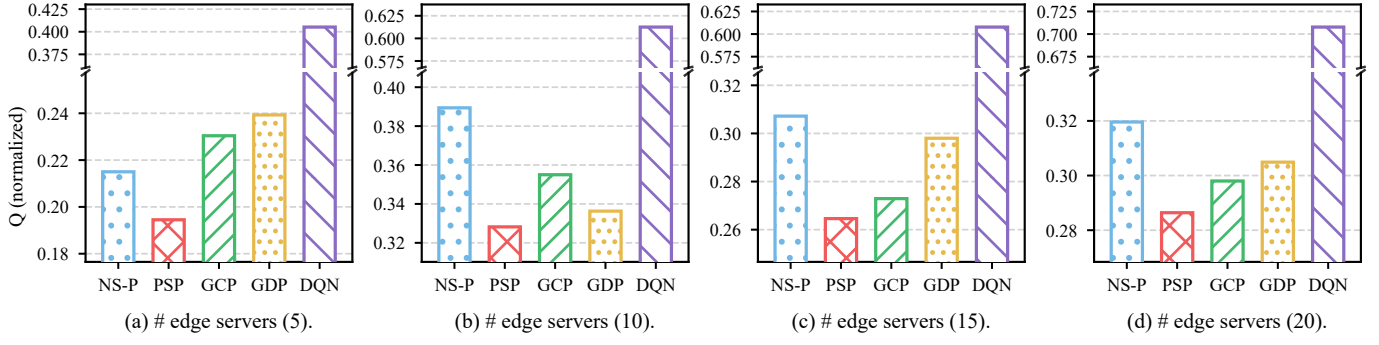


Fig. 7. Service-provisioning performance with 130 users and increasing numbers of deployed edge servers.

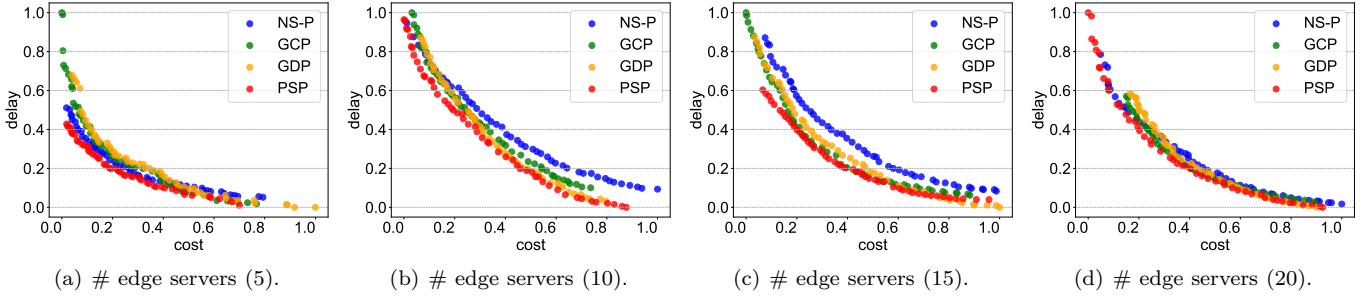


Fig. 8. Pareto fronts for Stage-II service provisioning with 130 users.

conducted on a workstation running Windows 11, equipped with an AMD Ryzen 7 5800H with Radeon Graphics processor, an NVIDIA GeForce RTX 3050 Ti GPU, and 32 GB of RAM.

The evolutionary settings used for PSP are summarized in Table II. The deterministic anchor size is defined proportionally as $\varpi_j = \lceil \rho V_j \rceil$ with $\rho = 0.5$. Hence, for the experimental capacity $V_j = 4$, two slots retain the highest-scoring services and two slots preserve stochastic exploration. This equal allocation provides an interpretable exploitation–exploration balance and applies directly to servers with different capacities.

For each server/user configuration, cost and delay are normalized over the pooled solutions of the compared methods as

$$\hat{C} = \frac{C - C_{\min}}{C_{\max} - C_{\min}}, \quad \hat{D} = \frac{D - D_{\min}}{D_{\max} - D_{\min}}, \quad (15)$$

and the scalar evaluation score is

$$Q = \lambda \hat{C} + (1 - \lambda) \hat{D}, \quad \lambda = 0.5. \quad (16)$$

Lower Q indicates a better balanced solution. Hypervolume (HV) measures the dominated objective-space volume relative to the reference point (1.1, 1.1) and is maximized, whereas inverted generational distance (IGD) measures the mean distance from the common non-dominated reference front to a method's front and is minimized.

TABLE II
NSGA-II and PSP parameter settings.

Parameter	Value
Population size N	50
Maximum generations G	200
SBX crossover	$p_c = 0.9, \eta_c = 15$
Polynomial mutation	$p_m = 1/(k \mathcal{S}), \eta_m = 20$
Hybrid-score weights	$\alpha = \beta = 0.5$
Service capacity	$V_j = 4$
Anchor ratio	$\rho = 0.5$

In Stage I, CLS is compared with four baseline methods:

- Random Deployment (RD): k base stations are randomly selected as edge server deployment locations.
- Density Based Deployment (DBD): Base stations are ranked by user density within their coverage areas, and the top- k locations are selected for edge server deployment.
- Distance-Sum Deployment (DSD): For each base station, the cumulative distance to all users is computed, and the top- k locations with minimal total distance are chosen for edge server deployment.
- Greedy-K Deployment (GKD): The algorithm iteratively selects base stations that maximize the reduction in total user distance until k servers are deployed.

In Stage II, PSP is compared with four baseline strate-

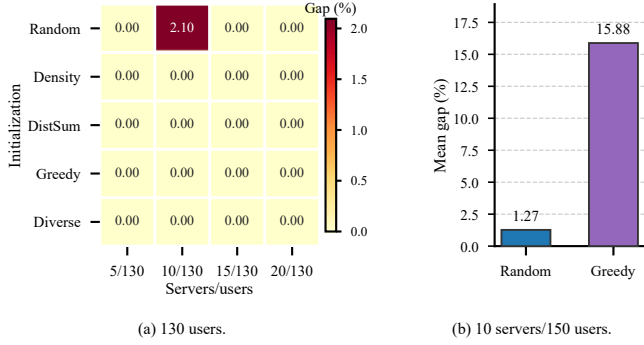


Fig. 9. Initialization sensitivity of CLS under two server/user settings.

gies:

- **NSGA-II Provision (NS-P):** The initial population is generated randomly.
- **Greedy-Cost Provision (GCP):** Services with the lowest deployment costs are prioritized until the capacity limit is reached during initial population generation.
- **Greedy-Delay Provision (GDP):** Services with the highest request volumes are prioritized until the capacity limit is reached during initial population generation to minimize user delay.
- **Deep Q-Network Provision (DQN):** A Q-network sequentially selects a service or an empty action for each server slot using demand, deployment cost, and a cost-delay preference. The resulting deployment is evaluated by the same objectives as the other methods.

B. Experiment Results

1) **Stage I: Server Deployment:** To examine the effect of the initial deployment set in Algorithm 1, we compare Random, Density, DistSum, marginal Greedy, and Diverse initialization over 50 runs. For each configuration, the reported gap is the percentage difference between the final cost and the best final cost observed under the same server/user setting. As shown in Fig. 9, the five strategies reach the same best final cost in nearly all fixed-130-user cases; the only nonzero entry is the 2.10% mean gap of Random at 10 servers. Under the 10-server/150-user setting, Random obtains a 1.27% mean gap, whereas marginal Greedy reaches 15.88%. These results show that CLS is generally stable across initializations and that random initialization avoids a systematic preference for a poorer local optimum.

We design two sets of experiments to demonstrate the superiority of our deployment strategy.

First, when determining the optimal number of servers k , we evaluate six values: the optimal k , $k \pm 2$, and the

maximum deployment capacity $\mathcal{K}^c$. These values are input into our cost model that jointly optimizes server deployment cost and communication cost. As shown in Fig. 4(a), our strategy achieves the lowest cost consumption at the determined k compared to neighboring values, demonstrating simultaneous optimization of both deployment and communication costs across varying server counts. Furthermore, Fig. 4(b) illustrates the convergence behavior when $k = 10$, where the algorithm progressively reaches optimal states through multiple iterations, resulting in optimized communication costs.

Second, to validate the superiority of our CLS algorithm, we conduct comparative experiments deploying 10 and 20 servers to serve 180 and 300 users, respectively. Fig. 3 presents the communication cost comparison across different algorithms under varying base station and user scales, revealing two critical findings: (1) the CLS algorithm consistently maintains lower communication costs as data scales increase, while (2) other algorithms—particularly DSD and GKD—exhibit significantly higher costs when user/base station scales exceed certain thresholds. This degradation occurs because DSD and GKD overemphasize user distance metrics while neglecting density distribution, adapting to distant outliers at the expense of clustered nearby users. In contrast, our local search strategy mitigates this issue through iterative optimization, demonstrating superior robustness in large-scale scenarios.

2) **Stage II: Service Provisioning:** We evaluate Stage II under two complementary configurations: (i) 10 deployed servers with 100, 130, 150, and 180 users, and (ii) 130 users with 5, 10, 15, and 20 deployed servers. Figures 5 and 7 report the minimum normalized score Q obtained by each method. PSP achieves the lowest Q in every reported case across both experimental series. Its hybrid initialization consistently improves upon random NS-P and the single-criterion GCP and GDP initializations; PSP also achieves lower Q than DQN across all tested scales.

Figure 8 compares the cost-delay fronts generated by the four NSGA-II-based provisioning strategies with 130 users. Table III gives the numerical results for the 10-server setting. PSP attains the largest HV (0.9470), the smallest IGD (0.0016), and the smallest Q (0.3282), confirming both broader objective-space coverage and closer convergence to the common reference front. DQN produces a limited set of scalarized solutions rather than a population-based Pareto front; it is therefore compared through Q and is not included in the HV/IGD comparison.

VI. Conclusion

This paper presents MOS², a two-stage multi-objective framework for joint cost-delay optimization in mobile edge networks, addressing server deployment and service provisioning in multi-user scenarios. In the first stage, we

TABLE III
Pareto quality with 10 servers and 130 users.

Method	HV $\uparrow$	IGD $\downarrow$	Best Q $\downarrow$
NS-P	0.8191	0.0785	0.3894
GCP	0.8596	0.0492	0.3550
GDP	0.8945	0.0326	0.3363
PSP	0.9470	0.0016	0.3282
DQN	—	—	0.6125

develop a Coverage-based Local Search (CLS) algorithm that determines the number of deployable servers from budget and user-density constraints and then iteratively refines their placement to minimize deployment and transmission cost. Building on the resulting spatial configuration, the second stage formulates service provisioning as a bi-objective problem and solves it with a Pareto-oriented Service Provisioning (PSP) strategy on top of NSGA-II. To accelerate convergence and improve solution quality, we further design a standalone Initial Base Configuration (IBC) procedure that fuses cost-oriented and delay-oriented greedy rankings into a structured initial seed, and we embed an explicit capacity-repair mechanism into the evolutionary loop to enforce feasibility. Complexity analyses are provided for both stages, and extensive experiments on real-world base station data show that the framework consistently outperforms baseline methods and delivers a rich set of Pareto-optimal trade-offs between cost and latency, giving network operators practical flexibility in provisioning decisions. **Future work will extend the framework with reliability-aware QoS constraints for packet loss, link availability, and service interruption, together with learning-based demand prediction and multi-domain coordination.**

References

- [1] Wang S, Zhao Y, Xu J, et al. Edge server placement in mobile edge computing[J]. *Journal of Parallel and Distributed Computing*, 2019, 127: 160-168.
- [2] Li Y, Wang S. An energy-aware edge server placement algorithm in mobile edge computing[C]//2018 IEEE International conference on edge computing (EDGE). IEEE, 2018: 66-73.
- [3] Xu X, Xue Y, Qi L, et al. Load-aware edge server placement for mobile edge computing in 5G networks[C]//Service-Oriented Computing: 17th International Conference, ICSOC 2019, Toulouse, France, October 28–31, 2019, Proceedings 17. Springer International Publishing, 2019: 494-507.
- [4] Jiang X, Hou P, Zhu H, et al. Dynamic and intelligent edge server placement based on deep reinforcement learning in mobile edge computing[J]. *Ad Hoc Networks*, 2023, 145: 103172.
- [5] Kasi S K, Kasi M K, Ali K, et al. Heuristic edge server placement in industrial internet of things and cellular networks[J]. *IEEE Internet of Things Journal*, 2020, 8(13): 10308-10317.
- [6] Wu J J, Shih S F, Liu P, et al. Optimizing server placement in distributed systems in the presence of competition[J]. *Journal of Parallel and Distributed Computing*, 2011, 71(1): 62-76.
- [7] Shen B, Xu X, Qi L, et al. Dynamic server placement in edge computing toward internet of vehicles[J]. *Computer Communications*, 2021, 178: 114-123.
- [8] Ling C, Feng Z, Xu L, et al. An edge server placement algorithm based on graph convolution network[J]. *IEEE Transactions on Vehicular Technology*, 2022, 72(4): 5224-5239.
- [9] Luo F, Zheng S, Ding W, et al. An edge server placement method based on reinforcement learning[J]. *Entropy*, 2022, 24(3): 317.
- [10] Zeng F, Ren Y, Deng X, et al. Cost-effective edge server placement in wireless metropolitan area networks[J]. *Sensors*, 2018, 19(1): 32.
- [11] Guo Y, Wang S, Zhou A, et al. User allocation-aware edge cloud placement in mobile edge computing[J]. *Software: Practice and Experience*, 2020, 50(5): 489-502.
- [12] Xu Y, Chau V, Wu C, et al. Online joint placement and allocation of virtual network functions with heterogeneous servers[J]. *IEEE Internet of Things Journal*, 2020, 7(9): 8049-8058.
- [13] Cui G, He Q, Chen F, et al. Trading off between user coverage and network robustness for edge server placement[J]. *IEEE Transactions on Cloud Computing*, 2020, 10(3): 2178-2189.
- [14] Qian Y, Hu L, Chen J, et al. Privacy-aware service placement for mobile edge computing via federated learning[J]. *Information Sciences*, 2019, 505: 562-570.
- [15] Yu N, Xie Q, Wang Q, et al. Collaborative service placement for mobile edge computing applications[C]//2018 IEEE Global Communications Conference (GLOBECOM). IEEE, 2018: 1-6.
- [16] Ma H, Zhou Z, Chen X. Predictive service placement in mobile edge computing[C]//2019 IEEE/CIC International Conference on Communications in China (ICCC). IEEE, 2019: 792-797.
- [17] Ouyang T, Zhou Z, Chen X. Follow me at the edge: Mobility-aware dynamic service placement for mobile edge computing[J]. *IEEE Journal on Selected Areas in Communications*, 2018, 36(10): 2333-2345.
- [18] Ouyang T, Li R, Chen X, et al. Adaptive user-managed service placement for mobile edge computing: An online learning approach[C]//IEEE INFOCOM 2019-IEEE conference on computer communications. IEEE, 2019: 1468-1476.
- [19] Zhang Q, Zhu Q, Zhani M F, et al. Dynamic service placement in geographically distributed clouds[J]. *IEEE Journal on Selected Areas in Communications*, 2013, 31(12): 762-772.
- [20] Skarlat O, Nardelli M, Schulte S, et al. Optimized IoT service placement in the fog[J]. *Service Oriented Computing and Applications*, 2017, 11(4): 427-443.
- [21] Wang S, Urgaonkar R, He T, et al. Dynamic service placement for mobile micro-clouds with predicted future costs[J]. *IEEE Transactions on Parallel and Distributed Systems*, 2016, 28(4): 1002-1016.
- [22] Wang L, Jiao L, He T, et al. Service placement for collaborative edge applications[J]. *IEEE/ACM Transactions on Networking*, 2020, 29(1): 34-47.
- [23] Cabrera C, Svorobej S, Palade A, et al. MAACO: A dynamic service placement model for smart cities[J]. *IEEE Transactions on Services Computing*, 2022, 16(1): 424-437.
- [24] Yang L, Cao J, Liang G, et al. Cost aware service placement and load dispatching in mobile cloud systems[J]. *IEEE Transactions on Computers*, 2015, 65(5): 1440-1452.
- [25] Chen L, Xu J, Ren S, et al. Spatio-temporal edge service placement: A bandit learning approach[J]. *IEEE Transactions on Wireless Communications*, 2018, 17(12): 8388-8401.
- [26] Han P, Liu Y, Zhang X, et al. Energy-Efficient Service Placement Based on Equivalent Bandwidth in Cell Zooming Enabled Mobile Edge Cloud Networks[J]. *IEEE Transactions on Vehicular Technology*, 2022, 71(11): 12275-12290.
- [27] Sami H, Mourad A, Otrok H, et al. Demand-driven deep reinforcement learning for scalable fog and service placement[J]. *IEEE Transactions on Services Computing*, 2021, 15(5): 2671-2684.
- [28] Lu S, Jin X, Wu J, et al. QoS-aware Dynamic Service Caching and Updating in Cost-efficient Multi-Access Edge Computing[C]//2024 IEEE International Symposium on Parallel and Distributed Processing with Applications (ISPA). IEEE, 2024: 1718-1725.

- [29] Fang J, Wu S, Lu S, et al. Enhanced Profit-driven Optimization for Flexible Server Deployment and Service Placement in Multi-User Mobile Edge Computing Systems[J]. *IEEE Transactions on Network Science and Engineering*, 2024.
- [30] Lu, S, Xiang, B, Wu, J, et al. Multi-Timescale Hierarchical Prefetching for Online Caching in Vehicular Edge Networks[C]. //In 2025 34th International Conference on Computer Communications and Networks (ICCCN) (pp. 1-9). IEEE.
- [31] Lu S, Jin X, Wu J, et al. Enhanced Multi-Stage Optimization of Dynamic QoS-Aware Service Caching and Updating in Mobile Edge Computing[J]. *IEEE Transactions on Network and Service Management*, 2025.
- [32] Liu, H, Xu, Z, Di, X, et al. Multi-Stage Pareto-Based Optimization for Joint Server Deployment and Service Provisioning in Mobile Edge Computing[C]. In 2025 IEEE International Symposium on Parallel and Distributed Processing with Applications (ISPA) (pp. 363-370). IEEE.
- [33] C. Feng, M. Yang, Z. Jing, T. Q. S. Quek, and M. Mei, "Latency-Aware Service Deployment and Peer Offloading: A Long-Term Optimization Framework for Satellite Edge Computing," *IEEE Internet of Things Journal*, vol. 13, no. 1, pp. 405–417, 2026.
- [34] M. Kato, T. K. Rodrigues, and S. Verma, "Novel Breakout Local Search for Offloading Tasks in Multi-Tiered Cloud Environment Considering Transmission and Processing," in *Proc. 2025 9th IEEE International Conference on Network Intelligence and Digital Content (IC-NIDC)*, 2025, pp. 269–274.
- [35] Z. Jing, Q. Yang, M. Qin, J. Li, and K. S. Kwak, "Long-Term Max–Min Fairness Guarantee Mechanism for Integrated Multi-RAT and MEC Networks," *IEEE Transactions on Vehicular Technology*, vol. 70, no. 3, pp. 2478–2492, 2021.
- [36] Z. Jing, X. Wang, Q. Yang, M. Mei, and Y. Wu, "Dynamic Energy Cost Conservation for Distributed Edge Clouds Utilizing Online Mini-Batch Learning," in *Proc. IEEE 34th Annual International Symposium on Personal, Indoor and Mobile Radio Communications (PIMRC)*, 2023, pp. 1–6.